

SPECIAL ISSUE PAPER

The changing balance of technology and process: A case study on a combined setting of model-driven development and classical C coding

Regina Hebig^{1,2}  | Jesper Derehag³¹Chalmers University of Technology, Gothenburg, Sweden²University of Gothenburg, Gothenburg, Sweden³Ericsson AB, Gothenburg, Sweden**Correspondence**Regina Hebig, Computer Science and Engineering Department, Chalmers University of Technology, Gothenburg, Sweden.
Email: hebig@chalmers.se**Abstract**

The increasing flexibility in industry leads to an ecosystem of change, affecting the balance of processes and technology as well as the developers who have to cope with the change. Furthermore, the change itself might impact the ability to use quantitative methods to learn from previous experience. The goal of this study is to better understand the ecosystem of mutual impacts and changes of process and technologies as well as how developers perceive a technology setting and deal with its change. Therefore, we conducted a case study at Ericsson, performing a series of interviews among 6 employees (senior developers and architects). We identified a time line of changes that happened over 7 years. A set of observations about the relation between processes and tooling, and observations about developer's perceptions of the technology settings, and their strategy to deal with these changing technology settings. We discuss how the observed change impacts the ability to perform quantitative evaluations of technology and processes. The findings show that a bad choice of technologies can lead to unexpected impact on team dynamics. Furthermore, change happens so regular that it needs to be considered when collecting data for a quantitative evaluation of, eg, productivity.

KEYWORDS

case study, development technologies, empirical investigation, software process

1 | INTRODUCTION

Today, it is well understood that processes are not fixed constructs and need tailoring to fit the needs of a company or project.¹ Furthermore, we know today that processes change with time, which can not only be due to changing needs but also due to hopes that developers or management put into the new processes. Thus, companies want to be able to freely choose their processes and team structures.

This observation holds even for companies working in the embedded domain. As stated by Sam Nicander from SAAB* during a presentation in fall 2015[†], companies require tool-independent methodologies, ie, tools should not restrict or enforce the process anymore. In addition, it is known that used technologies change over time.²

One aspect that is clear today is that changes in technologies and processes can trigger each other, until a balance, ie, a setting where both can be applied together, is reached.³ In context of a company, this succession of changes cannot be without consequences. On the one hand, there are the developers who not only need to deal with the change but also have their opinions about technologies and methods. On the other hand, companies are interested in the question what impact the changes actually have.

Problem However, little is known about all these aspects. We still do not completely understand how development technologies such as code generation and modeling impact the process and team interaction.⁴ Also the question how developers deal with these changes is barely studied. Thus, we formulate 2 main research questions for this paper:

1. How does the balance of process and technology look like?
2. Do developers have strategies to deal with changes?

* SAAB <http://saabgroup.com/>[†] Sam Nicander, "Towards MBE - a tale of 4 introductions," Ericsson Modeling Day 2015

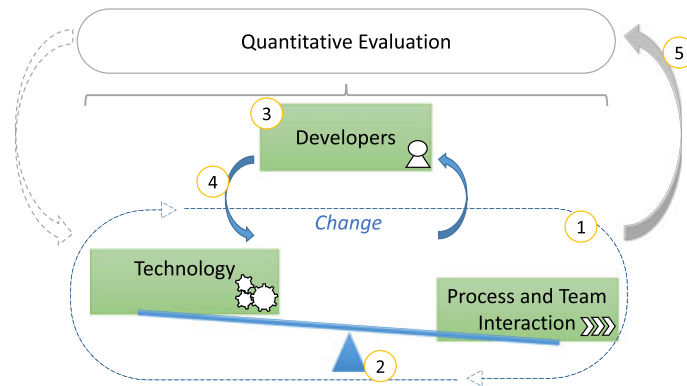


FIGURE 1 Technology and process/team interaction can trigger mutual changes. Continuous changes in both, however, might impact the ability to judge about the development environment

Objective To learn more about these aspects, we performed a qualitative study with Ericsson.[‡] We considered a specific product which parts are built with the help of 2 alternative settings of development technologies. The situation is the result of a history of changes. This constellation has the major advantage that the same developers have often worked with both settings and experiences them within the same conditions, including corporate culture and organizational structure.

Contribution For the study, we interviewed 6 employees of Ericsson (senior developers and system architects).

First of all, we were able to collect information about changes in context of both settings of used technologies, ie, concerning the technologies themselves, partially surrounding techniques and the applied process, including the timing of activities and iterations as well as the team interaction (1 in Figure 1). This way we could come up with a time line that shows how regularly the studied development environment was subject to change.[§]

Second, the collected data allowed us to make 6 observations on the relationship of development technologies and process or team interaction, which allows us to learn about their balance (2 in Figure 1). We not only could confirm some observations, which were made in related studies, but also uncover some so far unknown relations between technologies and processes.

Third, while there is clearly a process of balancing, it is not always clear whether the balance leads to an ideal result.³ To better understand that, we investigate any opinions that developers would have about the 2 technology settings (3 in Figure 1).

Fourth, we were interested whether developers have strategies to actively deal with the change (4 in Figure 1).

Finally, the studied company always strives for the possibility to base their decision making about future changes on previous experiences and on quantitative data. However, change itself can impact the data in a way that makes such analysis difficult. Therefore, we aim at discussing the impact of the collected change history on the ability to address a list of quantitative questions posed by the company (5 in Figure 1).

The paper is structured as follows. In Section 2, we give a short introduction to related work. Afterwards, we provide background

information on the investigated continuous development and maintenance of a product from Ericsson (Section 3). In Section 4, the method of our investigation is described. Identified observations as well as the time line is presented in Section 5. We discuss the results in context of related studies in Section 6 and threats to validity in Section 7. Finally, we conclude in Section 8.

2 | RELATED WORK

In this section, we discuss related work about change in software development techniques and processes, the relations of processes and technologies, as well as advantages and disadvantages of development with software models.

2.1 | Change in software development techniques and processes

In our own previous work, we studied change histories of 13 technical settings involving modeling techniques.² There are several other works studying changes in software development techniques and processes.

Works that deal with the changes of software processes have been presented for example by Diaz et al, who study a process improvement within Motorola;⁵ Ocampo et al, who studied how process standards evolve;⁶ or Kuhrmann et al, who made a systematic mapping study on software process improvement.⁷ Unterkalmsteiner et al performed a systematic literature review on approaches to evaluate the impact of software process improvement.⁸

Furthermore, there is the work of Kuhrmann et al who summarize how the same standard process was adopted, adapted, and evolved within different organizations to fit their needs.⁹

Few works focus on change in the development technologies. For example, Herrmannsdoerfer et al¹⁰ or Durisic¹¹ study changes on the level of modeling languages and standards, focusing on the different metamodel changes that happen.

2.2 | Process and technologies

The question how processes might be impacted by development technologies is one asked from different perspectives.

First of all research on process tailoring asks the question when and how processes need to be adapted.^{1,12} As summarized in a survey by

[‡]Ericsson <http://www.ericsson.com/>

[§]Please note that this time line is within the scope of the studied development settings and system. Thus, it does not include other changes that happened within Ericsson.

Kalus et al, it was found that the tool infrastructure can have an impact on this tailoring.¹

From the perspective of development technologies, the impact on processes was mainly investigated with regard to the use of modeling. The knowledge that technology surrounding modeling can have an impact on processes and teams have been around for a long time. Several proposals for modeling-specific processes (eg, Kulkarni et al,¹³ Loniewski et al,¹⁴ or Asadi and Ramsin¹⁵) show not only that there seems to be an impact but also that there is little systematic knowledge about that impact.⁴

Some studies have been published that had their main focus on modeling in software development, but provided on the side some observations on the relation between the used technologies and the process. For example, Aranda et al investigated a case study with General Motors where they found that modeling technologies can impact the division of work between different roles.¹⁶ Two further related case studies have been published by Heijstek et al who identified 14 factors in the use of model driven engineering (MDE) that might influence the process¹⁷ and Staron et al who reported on a possible effects on process tailoring.¹⁸

Burden et al and Whittle et al compared different case studies to identify impacts of modeling. For example, both studies led to the insight that modeling can result in a reduction of outsourcing.^{19,20}

Finally, we investigated in our previous work 2 cases that had histories of changing settings of development technologies (using modeling), and we identified several cases of coevolution between the technologies and the process, which allowed us to conclude on some interrelations between both technologies and processes.

Considering change in software development, some studies have been collecting information on different aspects such as the process or tooling. For example, change of processes had been studied by Nikitina et al who investigated the case of a company that moved from Scrum to Kanban.²¹ Similarly, Diaz et al investigated process improvement at Motorola.⁵

Other studies investigate changes in the context of technologies. For example, Karaila studies changes in the tool environment of the function block language.³² Other studies address changes in languages, such as Herrmannsdoerfer et al who study typical changes in modeling languages.¹⁰ Finally, we studied in our own previous work evolution histories of MDE settings, ie, settings of development technologies that include models.²

2.3 | Investigations about advantages and disadvantages of model-related tools

Whittle et al present data from 39 interviews and create based on that a taxonomy "of factors which capture how MDE tools impact MDE adoption."²² This taxonomy is split into categories, such as tool features, complexity, or human factors.

Hutchinson et al present an empirical study in which they assess disadvantages and advantages of model usage.²³ They summarize, based on a literature study, that modeling can increase the time to develop code. Furthermore, they find that communication is improved and that there is often a dissatisfaction with tools.

Mohaghegi et al perform a metastudy on 25 research papers.²⁴ They found several reasons for the application of MDE, eg, increased

productivity, improved quality, and improved communication. However, they conclude that the evidence for these advantages is still very thin.

Forward and Lethbridge performed a survey amongst 113 software practitioners.²⁵ They collected several disadvantages that concern modeling and code centric approaches. For example, their practitioners reported that code-centric development leads to complex code and misses a high-level view. In contrast, they found that modeling tools are not well integrated with programming environments and that traceability is missing.

Staron studies 2 companies that were in the process of adopting MDE, one of them successfully and one of them not.²⁶ By comparing these 2 cases, he extracts several conditions for adopting model-driven approaches. Amongst these conditions are tool maturity and process compatibility.

Finally, Kuhn et al performed an interview study to identify the impacts of model-driven development (MDD) introduction to an automotive developer.²⁷ They identify problems with tooling concerning managing parallel versions. Furthermore, in the case they observed, they saw a lack of proper tooling for maintaining traces. Finally, Kuhn et al. found that there is a need for shorter cycles in development.

3 | BACKGROUND—THE ERICSSON CASE STUDY

Ericsson is one of the biggest telecommunication companies. The case study that we investigated for this paper is the continuous development and maintenance of a product from the Ericsson portfolio to which hundreds of developers are assigned.

What makes the product interesting for this investigation is the fact that multiple settings of development technologies are used in parallel. The two most important of those settings are referred to as "classical C" and "MDD." Both settings concern the implementation part only and are embedded into the same framework of requirements and testing technologies. Thus, we hoped that the direct comparison of those 2 development settings allows us to identify differences in the process and team interactions.

Figure 2 shows the setting of development technologies for the classical C case. The 2 tools that are used for the formulation and management of requirements are thereby IBM Rational Rhapsody and IBM Rational RequisitePro. Further, PowerPoint is used to slice requirements down to units of smaller projects that serve as input for the development teams. Functional verification tests are written in TTCN, and an internally developed tool is used for nonfunctional verification. For the development itself, Eclipse is usually used as an integrated development environment (IDE). The code is mainly written in C and partially in C++. In design-level testing, it is divided between an internal black box testing framework and a smaller amount of unit tests. Furthermore, it can occasionally happen that ad hoc visualizations of class relationships and state machines are generated via Eclipse plug-ins.

Figure 3 shows the setting of development technologies for the MDD case. It can be seen that some parts of the setting are the same as for the classical C case. However, instead of Eclipse the tool IBM Rational Rhapsody is used as an IDE. Within Rhapsody, sequence diagrams, class diagrams and state machines are used. Out of these, class

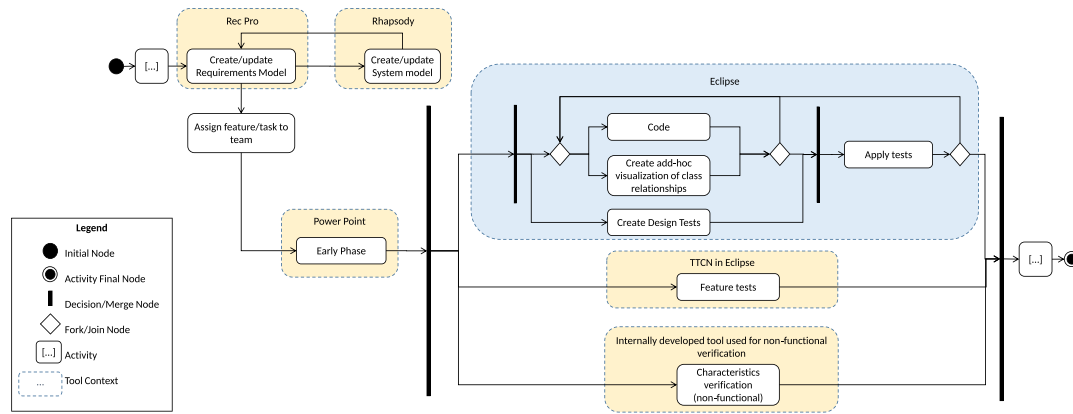


FIGURE 2 Setting of development tools for the classical C case

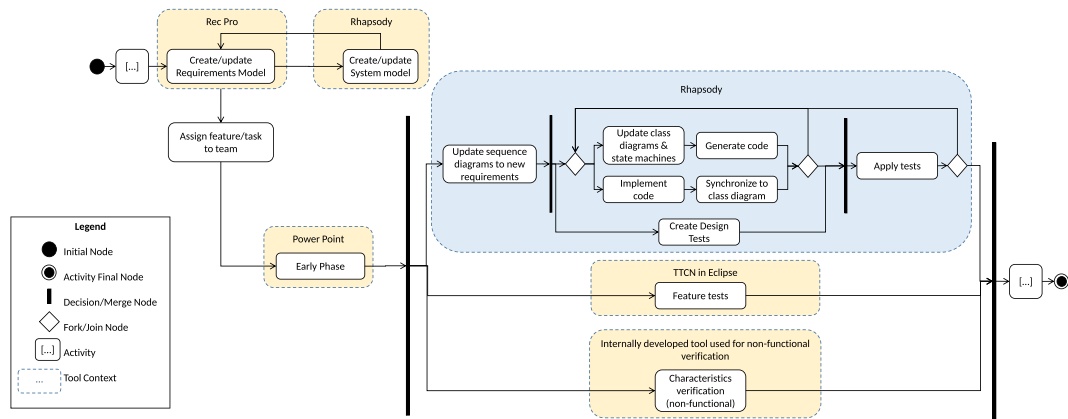


FIGURE 3 Setting of development tools for model-driven case (model-driven development)

and state-machine diagrams are used to generate C++. Worth noting still is that method bodies are still manually written and thus becomes a part of the model. Manual changes in the generated code can be round-tripped back to the models. In addition, the use of both unit and component tests (also developed using MDD) happens on a regular basis.

Another aspect that was already apparent when we started this investigation was that there had been some changes to the development technologies over the years. We decided to keep this in mind during the study, since experienced developers might be capable to draw comparisons between the respective old and new setting.

4 | METHOD

In this section, we describe the method that was used to investigate the case.

4.1 | Initial data collection and analysis method

Exploratory case study research²⁸ is ideal for the identification of so far unknown interrelations between development technologies and processes or team interaction, respectively. We decided to approach this case study with the help of semistructured interviews.²⁹ These allow us, on the one hand, to ensure a certain comparability between the interviews; on the other hand, this technique permits the interviewee

to bring in own topics and aspects. This is what allows us to actually find new insights.

However, the interview technique in itself is not sufficient to ensure a high quality and validity of the insights. This is first due to the reason that there is always a communication gap between interviewee and interviewer, eg, due to company internal wording that is not known to the external interviewer. Second, an interview of one person can only reflect the often partial and sometimes subjective perspective of this one interviewee. Thus, it is to be ensured that misunderstandings are minimized and that the subjectivity of the insights can be reduced.

A technique that permits that is grounded theory.³⁰ Grounded theory includes rounds of feedback with the interviewees. Conclusions that are drawn from comparing interviews with multiple experts are thereby fed back to the interviewees to give them the chance to confirm, correct, or even refuse the presented insights.

Our approach is inspired by the grounded theory. We performed individual interviews with each interviewee. Because of availability issues we could only perform a second interview with one of the interviewees. However, we had a final joined feedback interview during which all interviewees were present. The interest behind this joined interview was to see whether the interviewees have similar or diverse opinions about any insights. Further, this setting gave them room to discuss these insights immediately, which was quite well received.

To analyze the records of the interview and draw the insights that could then be subject to refinement, we used the constant comparison

method.²⁹ In this method, transcripts of interview records are annotated with semantic codes. These codes can then be used to identify and compare parts of different interviews that belong to the same topic.

In our case, these codes helped especially to first distinguish between parts of interviews that concerned

- changes in the setting of development technologies and processes over time,
- impacts of the setting of development technologies on processes and team interaction, and
- opinions on advantages and disadvantages of the development technologies and processes.

We performed 2 rounds of coding. In the first round, we went through the interviews and applied simple codes, such as “process”, “tool”, “change”, or “problem”. During that first round, we worked on refining the codes, eg, refined change to “process change” or “tool change”. The interviews were analyzed a second time with these refined codes. The, hereby, drawn insights were presented in the final joined meeting to all interviewees at once.

4.2 | Selection of subjects

The study was organized by the 2 authors taking different roles. While one of us performed the interviews and analyzed the data, the second of us was, being Ericsson internal, responsible for selecting and approaching potential interviewees.

That selection was done based on the experience of the interviewees. Therefore, we focused on interviewees that have worked in the MDD domain for a couple of years apart from any other experience. Thus, we ended up targeting senior developers and architects that not only represented the highest amount of experience but we also tried to get the maximum amount of variance between their general attitudes towards MDD.

Altogether, we had 6 interviewees, most of them with approximately 10 years or more experience in the industry. Each interviewee worked at the time of the interviews at least 4.5 years (and at most 10 years) on the studied system.

4.3 | Study execution

Interviews were planned to take approximately 60 minutes. However, due to interesting conversations some of the interviews were spontaneously prolonged and took up to 90 minutes. The interviews per interviewee took place in November 2014. The final joined interview was performed on February 6, 2015.

Following topics were used as interview guidelines to structure the interview:

- background questions on interviewees experience;
- process-related questions, eg, we asked participants to describe the process from their perspective;
- the settings of development technologies;
- experienced differences between the settings of development technologies, eg, we asked for differences in
 - a. the development activities performed,
 - b. the timing,

- c. quality assurance activities, and
- d. the team structure;

- the interviewees personal opinion about advantages and disadvantages of the different settings of development technologies and the process, eg, we asked the interviewees to name 3 things they would want to change;
- after the first interviews were done, we added the question whether the interviewee remembered changes that happened to the setting of development technologies or processes.

4.4 | Quantitative questions

As indicated in Section 1, we collected in addition to the interviewees' personal opinions also a list of quantitative questions that the company would have liked to answer to compare both technology settings.

We had access to this list due to a long active research collaboration between Chalmers and Ericsson. The list was indeed already presented to a team of several researchers in Chalmers around the time of the interviews.

To approach the question whether and to what degree the observed changes impacted the ability to use the questions from that list, we investigate each question regarding the required data and whether this data could have been impacted by the change.

5 | RESULTS

In this section, we summarize the insights identified during the study. We follow the structure of Figure 1, starting with (1) the time line of identified changes, followed by (2) the aspects of the balance between technologies and process and team interaction, (3) developers' perceptions, and (4) a found strategy that developers use to cope with changes. Finally, (5) we shortly present the identified quantitative questions that were of interest for the company.

5.1 | Identified changes

One major observation was the huge number of changes that happened regarding both the process and the development tools. So we decided to draw more attention to these changes during the interviews and were able to collect a list of changes that happened over the years.

Note that it was not possible to reconstruct precise dates of the changes. However, we could place the changes roughly by year and also could retrieve some partial orders between different changes. The resulting time line was shown to all interviewees in the joined final interview and (after 2 slight corrections) confirmed by the interviewees.

Figure 4 and Table 1 summarize the time line. We identified 13 changes that can be split into the following 4 groups.

Model-driven development changes We found 2 changes that were specific to the development tooling of the MDD part. Around the beginning of 2009 the MDD tool Rhapsody was introduced together with the modeling language UML and a code generator that can create C++ code from the models. That setting of MDE/MDD development technologies was

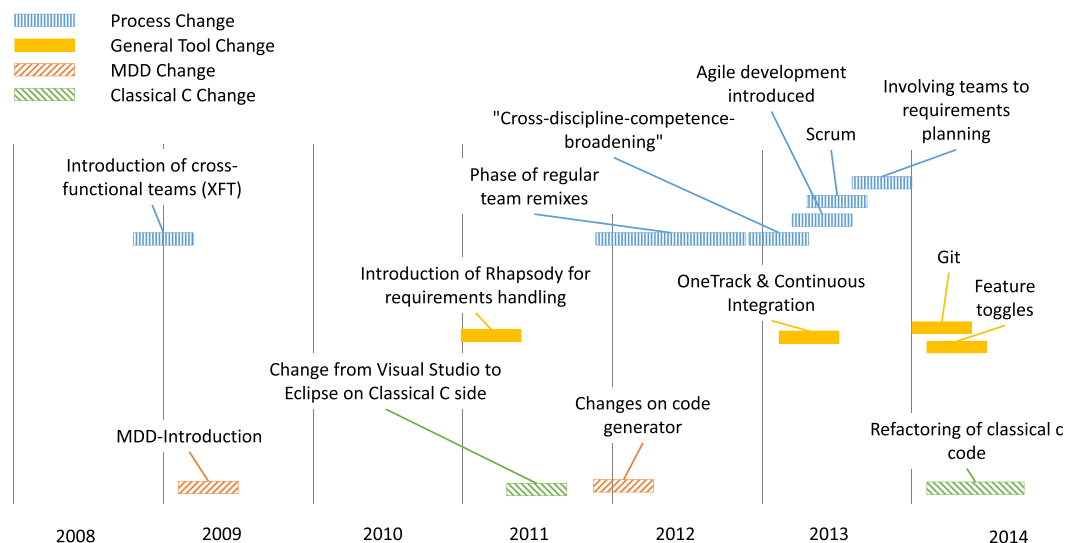


FIGURE 4 Time line of changes in context of the studied development settings (model-driven development [MDD] and classical C)

TABLE 1 Summary of changes and the motivations behind them

Change	Description	Time of Change	Motivation for Change
M1	introduction of the MDD tooling (Rhapsody) and modeling	start of 2009	hope for better code structure
M2	exchanged code generator	2011/2012	performance increase
C1	change from Visual Studio to Eclipse	end of 2011	
C2	C to C++	ongoing since 2014	testability
G1	requirements in Rhapsody	after 2010	
G2	from ClearCase to Git	start 2014	scalability, costs
G3	OneTrack & Continuous Integration	start of 2013	prevent fault resurfacing
G4	Feature toggles	start of 2014	enhance visibility
P1	cross-functional teams	2008/2009	shorten handovers
P2	regular team remixes	from end of 2011 to end of 2012	prevent idle times
P3	"cross-discipline-competence-broadening"	2012/2013	stabilize teams
P4	agile development	early 2013	better integration of activities
P5	Scrum	midst of 2013	standardization of agile processes
P6	integration of teams to requirements planning	end of 2013	costly handover

Abbreviation: MDD, model-driven development.

initially planned and set up to substitute the classical C development. When the need for a new component arose, Ericsson decided to redirect the use the MDD setting for developing that new component instead of continuing using the old paradigm.

A second change was made between 2011 and 2012: the code generator was optimized to reach a better generation performance. The generation time for was reduced from 2 hours to 10 minutes.

Classical C changes End of 2011, the IDE for the classical C development was changed from Visual Studio to Eclipse.

A change that started in 2014 and was at the time of the interviews ongoing is the refactoring and redesign of the classical C code. To enable more unit tests and decrease the coupling within the code, the classical C code was redesigned to object-oriented C++ code.

General tool changes After 2010, the requirements documentation was changed so that requirements were now written in Rhapsody. Test are tagged with references to these requirements.

At the beginning of 2014, the version control system was changed from ClearCase to Git to reach a better scalability and to save costs.

So as to prevent that already resolved faults resurface when new features had to wait longer times for delivery, OneTrack and continuous integration were introduced at the beginning of 2013.

In 2014, the concept of "feature toggles" was adopted to allow teams to deliver large nonenabled features to the master branch. The main goal was to avoid big-bang integrations, especially for the MDD part but also to enhance visibility of features that are still in development.

Process changes Between 2008 and 2009, the team structure was changed to mixed teams or “cross-functional” (XFT) teams were introduced. Before that designers and testers had not been in the same teams, which led to long handovers. Two years later, a 1-year phase (from end of 2011 to end of 2012) followed in which teams were regularly remixed. The motivation behind that was to prevent idle times, eg, when projects included a lot of testing, developers would have less to do. In such cases, teams were expanded with more testers. So as to regain team stability, this phase was ended with the adoption of another solution for dealing with idle times: the “cross-discipline-competence-broadening.” Thus, team members were motivated to broaden their skills to help their teammates out. For example, developers would help out with simple testing tasks when the test load was high. So as to deliver more closely what the customer wants, an agile development style was adopted in early 2013. A bit later (around midst of 2013), the next step in the development teams was the introduction of Scrum. Finally around the end of 2013, the process was changed so that teams are more involved into requirements and feature planning. Before that the inputs for the teams were very simple, which made the handover overall costly.

5.2 | The balance of technology on process and team interaction

The case study at hand allowed us for 2 reasons to draw conclusions on the interrelations of development technology and process or team interaction. First, the case study included alternative settings of development technologies—which allowed for a comparison. Second, the technologies of the case study were subject to a multitude of changes in the last years. In consequence, the interviewees were enabled to reflect on changes and the impacts on team interaction they experienced. In the following, we present the observations that we made about technology impact on timing of iterations and roles.

Timing 1: Code generators Our first observation was that the performance of code generators can have an impact on the frequency with which developers decide to generate and compile the code.

While there are no hints that a code generator with a good performance impacts the frequency of these decisions, we found several hints that a bad performance, ie, a long generation time, impacts these decisions.

Interestingly, we observed 2 seemingly contradicting tendencies: On the one hand, there is the tendency that a long code generation time reduces the frequency of code generation and compilation. Thus, some developers try to save time by performing several changes before regenerating and testing the code:

“You are trying to avoid (to) build if you know that you have to add some lines in this part as well. You don’t really try to rebuild it immediately.” [interview transcript]

On the other hand, there is the tendency that a long code generation time increases the frequency of code generation. Some

interviewees reported that developers try to do the initial code generation as soon as possible. This would save time later on where one would only partially generate a model to speed up the feedback loop. During the joined feedback meeting the interviewees agreed that the first tendency of a reduced frequency seems to instantiate more often in practice.

We observed both tendencies on a model-based code generator, ie, a tool that takes models (in this case, UML³¹ class diagrams and state machines) and produces source code (in this case C++ code).

Timing 2: Partial code generation As indicated above, the model-based code generator allowed for a partial code generation. The second observation in this regard shows the importance that short iterations have for developers. We observe that due to the long code generation time, developers invest the effort to produce partial generations. These partial generations are quicker than the overall generation of the code. This way the developers can include additional short build and test cycles before they run the complete code generation and tests. As one of our interviewees put it

People who know how things are set up [...], I believe, they want to get down to definitively the minimum of ... make things as fast as they can. If they can by just building one thing get down to 30 seconds, they would for sure do that instead of waiting 5 minutes for everything [interview transcript]

However, the required configuration seems to imply extra effort. It is interesting to see that most developers seem to be willing to invest in these efforts when their ability to develop in short iterations are at risk. During the joined feedback interview, the interviewees confirmed that they would wish for an automated partial generation of code. Again, it would be interesting to see whether that observation holds in general for generators and compilers.

Timing 3: Merging tools Tools that enable merging of development artifacts are a core technology for joined development. However, depending on these artifacts, merging tools have diverse quality. Especially when it comes to the use of modeling, merging tools can have low quality, implying a high amount of conflicts that need to be resolved manually.

One observation we made is that this quality seems to have impact on the frequency in which single teams reintegrate their changes with the rest of the system. As for the first observations, we could identify 2 different tendencies here as well. On the one hand, a low quality of the merge tool seems to increase the rate of reintegrations. The reason for this is, that smaller changes reduce the probability of merge conflicts. On the other hand, it seems that in some cases a low quality of the merge tool leads to a reduced frequency of system integrations:

“We had to re-base infrequently. [...] Instead of doing it several times a week, we were just doing it only once every second week.” [interview transcript]

This seems to be the case when it is known that conflicts cannot be prevented. One of our interviewees reported about a big project where due to the size of the models, the process was adapted. While merges to the main branch are usually performed several times a week, this was slowed down to once every 2 weeks in that project. At the end, the main branch was locked for a few weeks and only this one project branch was integrated.

Timing 4: Sprint duration Finally, we made an observation on timing that can hardly be generalized to other situations directly. When being asked for which part of the development settings they estimate takes longer sprints, the interviewees responded that they plan more time for sprints with the model-based code generation. As reasons for that the “re-basing and delivery,” ie, the parts of the process where the code needs to be integrated with the overall system where mentioned. This fits to the above made observations that the respective tooling also has an impact on timing decisions. While this observation is first of all very specific to the 2 settings of development tools at hand, it is interesting. It shows that the impact of tooling on iteration lengths seems to exist not only on the level of smaller code, build, and test iterations but even also on the level of sprints.

Roles 1: Communication One often argues and hopes on the advantage of model usage is their support for communication. This case study now provides us with hints that this advantage is real and even has an impact on the question what decisions can be discussed and made together by different roles in the process.

Our interviewees reported that developers use these diagrams to discuss the planned structure of their code with colleagues.

“I can quite easily show it to someone sitting next to me, asking: ‘Is this making sense?’” [interview transcript]

Further, this up-front action of requesting feedback on planned design seems to be less often when no models are used.

Even more interesting, it was reported that models enable discussion between different roles. For instance, system managers who normally does not work on a code level and are consequently not involved in decisions during coding phase. However, because of the use of models, system managers got involved into decision making at that level.

Roles 2: Need for experts One observation we made regarding roles is that there is a huge difference between tools and languages used in the same process that seem to create the need for experts.

Because of the conceptual gap between the languages used for test specification and development, also leads to a gap between the roles of testers and development. In the Ericsson case, this is acceptable since a differentiation of these roles is at the moment an intended part of the organizational structure. However, our interviewees expressed the opinion that it would be difficult for a single person to be expert in both the development and the testing language.

A question that arises from this observation is whether good tooling and language designs can help to reduce the need to separate roles. One of the interviewees reported on a idea to generate the tests based on an architectural description. In his words

[...] It will kind of eliminate the roles, a little bit, because the feature tester will no longer be working with TTCN and will be an expert on TTCN and how the components work together. They will more be experts on how to describe behavior, using English, and maybe how to transform this. So some people will need to be experts on how the feature tests work, but some people could also work on turning those into component tests. So you should be able to cover a lot more test cases and have more brain-power doing this rather than trying to figure out the syntax for a TTCN test case. [...]” [interview transcript]

Thus, the interviewee’s hope is that this representation would at least ease the communication between the roles.

5.3 | Perceived comparison of the technology settings

In the following, we summarize how the interviewees perceive the 2 technology settings in comparison, including what they found advantageous.

5.3.1 | Why to prefer the MDD setting

We collected 7 reasons for which the MDD setting is perceived as advantageous.

- a. One disadvantage that was mentioned about the classical C setting is the missing connection between code and requirements model. It seems to be a challenge to locate the implementation of requirements within the code. Our interviewees further reported that this problem increased after a refactoring that was performed on the requirements model.
- b. Our interviewees did rate the applicability of unit tests in the classical C setting somewhat lower than the one of the MDE setting. One reason was the age of the observed system and that some parts, where not designed for unit tests from the beginning. But other types of testing were present in that setting that were used instead. However, some of the interviewees preferred unit tests to these other tests that were more black box in their characteristics.
- c. An aspect that was perceived as positive about the models was the actual visualization they provide. Our interviewees reported that they support communication and planning. In contrast, developers report that it is not as easy in the classical C setting to gain an overview about the system and that this in turn affects the way they are approaching the code.
- d. One advantage that was mentioned as positive about the MDD setting is that code and models stayed consistent, meaning that the

- developers could trust that these parts of the documentation stayed up to date. Our interviewees referred to this as the benefit of an all-in-one-tool.
- e. State machines were perceived as the most useful model. Amongst the interviewees, they were perceived as helpful and as a real raise in the level of abstraction. Especially, the ability to represent asynchronous aspects of the system with them made them appealing to the developers.
 - f. An indirect advantage of the MDD setting stems from the massive use of unit tests. Because of them having shorter feedback loops they were perceived as positive by our interviewees. As one of them put it "Actually, the feedback loops are the valuable thing. And if you have design tests [= unit tests] they are per definition almost always shorter." [interview transcript]
 - g. **the classical C setting** However, there are also reason for the interviewees to rate the classical C setting as advantageous than the MDD setting.
 - h. In large companies, such as Ericsson, parallel work of multiple developer teams is part of the daily business. Given that context, the general immaturity of merge support for modeling can become a disadvantage. Our interviewees experienced a lot of situations where manual rework was necessary within the MDD setting. This not only concerned mostly state machines but also sequence diagrams. An aspect that was perceived as especially difficult were merge conflicts that appeared due to rearranging elements within the model.
 - i. Some interviewees indicated that there was some occurrences of duplicate code within unit and component tests within the MDD setting. They reported that the duplicates seemed to stem from use of copy paste when creating new test cases and not in the actual product code and that some of these test cases were harder to maintain than the ones with less copy paste sections. However, it is unclear if this difference between the setting was due to tooling or some other reasons.
 - j. In direct comparison, the IDE used for the classical C setting outperformed the one used for the MDD setting. This later IDE was not only used to facilitate that model creation and code generation, but also for coding itself. However, while the model related functionality was in focus of the tool vendor the coding support stayed behind that of many standard IDEs. Namely, our interviewee complained about a bad navigation support and bad refactoring support.
 "After a while you get tired of all that clicking, pop-up windows and ... so I don't like that part of the tool ... —so I like the graphical part [...] the modeling itself is quite easy." [interview transcript]
 - k. As already indicated above, code generation within the MDD setting could sometimes take several minutes, which was perceived as negative by many interviewees.
 - l. **code generation** Another aspect of code generation in the MDD setting was a problem concerning breaking code, ie, manual code changes that get in conflict with later on generated code. This disadvantage was not perceived serious though, since the test and delivery structure within the company prevented these situations from affecting a wider range than small teams. According to our interviewees, broken code was never delivered outside teams.
 - m. Finally, some interviewees reported that the classical C setting had a less steep learning curve than the MDD setting. One of the interviewees perceived that it could take months to fully understand and use the MDD tool itself (not necessarily the code).
 - n. About some of the advantages of the MDD setting that the interviewees perceived, they thought that they could simply be adopted for the classical C setting, too.
 - o. One process requirement that was emphasized to be positive is the requirement for unit testability. Our interviewees reported that the existence of this requirement changes the mind-set of the developers and how they structure their code. While this was observed for the MDD setting, it seems to be not bound to the technical setting but transferable.
 - p. Finally, good object-oriented APIs was mentioned as a positive aspect of the MDD setting. However a big refactoring project which included transitioning to object orientation had been ongoing in the classical C setting for some time and thus those perceived advantages are therefore obviously transferable as well.

5.4 | Dealing with change

Changes in process and technologies affect developers who have to deal with the new situation. Two of the above reported changes concerned the introduction of a new document, such as new models or new documents for storing and communicating requirements. Both cases show that a strategy evolved within Ericsson that allows developers to adopt the change during an "incubation phase" and get up to speed working with the new documents efficiently.

The 2 reported incubation phases concerned the addition of a new requirements document and the addition of state machines and sequence diagrams:

Case 1: the optional document In the first case, the introduced models were optional, ie, they worked as an alternative to already existing options. This leads to the question when 1 document (eg, a model) should be used instead of another one (eg, code). How to answer this question seems to be subject of experience. As one of our interviewees puts it
 "[...] I think after a while you reach a point where it makes sense to you to do it as a state machine or you shouldn't do it as a state machine. [...]" [interview transcript]

Case 2: the mandatory document In the second case, documents are not optional in use, such as new requirements documents, can be filled/used in different ways. In the case study at hand, it took a longer tryout phase until the designers had a good idea of what level of abstraction/degree of detail would be appropriate for that document.

Developed strategy Most interestingly, the Ericsson employees have developed internal processes to pass on their experience. One interviewee reported that they partially use a mentoring system, where employees, who already have experience with a document guideline was paired with employees who use a document for the first time.

TABLE 2 List of quantitative questions

ID	Question	Metric
Q1	Is there a difference in defect density between prescriptive MDE and manual code?	defect density
Q2	Is there a correlation between defect density and unit/component test coverage?	defect density, test coverage
Q3	Is there a correlation between defect density and the amount and/or expertise of contributors?	defect density, number of contributors
Q4	Do continuous-integration lead to a (quantitatively) higher productivity?	software size, #features
Q5	Is there a difference in productivity between MDE and manual code?	software size, #features
Q6	Is there a difference between prescriptive MDE and manual code with respect to over engineering?	software quality, requirements
Q7	Is there a difference between object-oriented vs procedural vs functional programming with respect to over engineering?	software quality, requirements

5.5 | Quantitative questions

As explained in Section 4.4, we collected a row of quantitative questions that were formulated by the company around end of 2014 (summarized in Table 2). During several years of coexistence of both technology settings the company continuously collected data about their development, through the used version control systems. The list was created on the assumption that this data can be used to quantitatively approach the question whether different technologies are to be preferred over the others.

We can see that the questions target on quality (of defect density), productivity, and over engineering (ie, whether software is build in a more complex and robust way than required).

Furthermore, the questions are not just concerned with the 2 above described technology settings. They also concern techniques, such as unit/component tests and continuous integration. Furthermore, the impact of the expertise of developers seems to be of interest.

To sum up, we can see that the companies quantitative questions concern nearly all aspects of development that they might influence and change.

6 | DISCUSSION

In this section, we discuss our findings and analyze how change impacts quantitative investigations. Furthermore, set all findings into context of previously discussed insight from related studies.

6.1 | Change—Change is a permanent part of development

Considering the time line of changes, since 2009, there was only 1 longer period for which we identified no changes. None of us really expected the massive amount of changes. Even to our author who works in Ericsson, the full extent became only clear after we summarized the changes as a time line as shown in Figure 4.

One reason, why we did not expect this extent is that we took both changes in the process, teams, and the development technologies into account. However, the expectation we had was mainly based on existing studies, which in most cases either focus on process change, eg, Diaz and Sligo,⁵ Ocampo and Münch,⁶ and Kuhrmann et al,⁹ or change in the development technologies, eg, Herrmannsdoerfer et al¹⁰ and Hebig and Giese.²

A question that has to stay open here is whether this density of changes is just a phenomenon of the last years or if that recent changes are better remembered than changes that are 5 to 10 years away. Nonetheless, the assessed time line makes it clear that developers need to be able to deal with regular change in their working environment.

6.2 | The impact of technology on process and team interaction

Several related studies already observed that the use of modeling can change the interaction of roles. For example, Heijstek et al found that modeling can enable requirement engineers to create parts of a system.¹⁷ A similar observation was made by Aranda et al,¹⁶ who found that domain experts were suddenly enabled to take over parts of the software development work. A little bit weaker version of this observation was made by Burden et al, who observed that modeling at least enables experts and developers to work together.¹⁹ Our observations now confirm these findings.

Our second observation on the roles, that a big technical gap can enforce roles fits the observations that we did in our previous work. There we observed that technical change can lead to change in the role structure of teams.³ The now made observation goes a step further since it does not assign the split of roles to an effect caused by team dynamics and reactions to change but to the actual difference between used languages.

One of our observations on timing, confirms that development technologies can have an impact on the duration of sprints. Some related studies led to similar observations. For example, Kulkarni et al¹³ recognized this impact and introduced a metasprint for long running tasks to their agile process.

However, the other 3 of our observations on timing are on a much more fine granular level than mechanisms so far discussed in literature. It is interesting to see that the tooling can impact small iterations done by developers via multiple mechanisms.

Thus, the study allowed to to gain more insight into our research question RQ1.

6.3 | Perceived comparison of the technology settings

In the following, we compare the found advantages and disadvantages to findings from literature.

When comparing what our interviewees perceive as nonideal about their technology settings with related work it gets easily visible that all of these aspects are well known problems that are not specific for any company.

For example, also Kuhn et al observed problems with diff tools for models, similar to the observed merge problems.²⁷ In general many researchers report on a missing maturity of tool support for models.^{22,24,26}

Similarly, the found missing integration of modeling and coding within the used tool was found in a similar way by Forward and Lethbridge,²⁵ Hutchinson et al,²³ and Whittle et al.²²

However, it also seems that Ericsson managed to overcome several problems that are listed in literature. For example, Forward and Lethbridge report about problems with inconsistencies between models and code,²⁵ while our interviewees report that they perceive the consistency of both (models as documentation and code) as an advantage of their MDD setting.

Furthermore, Kuhn et al found that development cycles are too long, while, again in our case study some of the feedback cycles, such as the design testing, are perceived as advantageous short. The found improvement of communication is in line with other research of Hutchinson et al²³ and Mohagheghi and Dehlen.²⁴

Finally, our results show that several properties are transferable between approaches with and without modeling. Besides the 2 found advantages of the MDD setting that might easily be (and are) adopted to the classical C setting, there is another hint. In literature, the lack of traces is often held against model-driven approaches, eg, by Kuhn et al²⁷ and Forward and Lethbridge.²⁵ However, while we found a similar issue in our case study, it was not associated with the MDD setting but the classical C setting.

The found differences between literature and, here, studied case are mainly caused by the differences amongst 2 model-driven settings might be bigger than the ones that in general distinguish model-driven settings and classical coding settings.

6.4 | Dealing with change—A first step towards a solution pattern?

The first observations we made concerning RQ2 show that developers have strategies to deal with changes in processes and technology settings and to ensure that new knowledge is passed on.

In former works we already found that the challenge of dealing with new artifacts can be so big that it actually affects the team structure.³ Given that context the observation of a strategy that allows spreading the knowledge is exciting. Future work needs to explore whether the strategy developed in Ericsson can actually be extended to serve as a general solution pattern for this problem in context of other companies, too.

6.5 | Quantitative questions—Change histories limit the usability of quantitative data!

One common aspect of the collected questions is that they are ultimately targeting at relating an effect (eg, higher quality or productivity) with a cause (eg, a technical setting).

Given the fact that companies such as Ericsson have large data collections about their development, it is worth considering this data for a quantitative analysis. This does not only hold for the management within a company but also for researchers.

However, quantitative investigations easily suffer from confounding factors. Given the eventful change history of the Ericsson case study, we want to discuss to what degree these changes represent confounding factor to be considered, when using the collected data. The concerns the question from what point in time the assessed data can stem in order to provide insight into the question at hand.

Earliest point of study First of all it is to be considered that there might be a latency between the introduction of the technology under study and the time where its effect is visible. Thus, the earliest time from which data can be used for an analysis is not necessarily from the technology introduction.

One reason for this is the incubation time that might be required for developers to get up to speed with the new technology (as we observed in Section 5.4). For the questions at hand, this concerns for example productivity or also the question whether a product is over engineered. During such an incubation time, data will not be representative.

The second reason is that it might simply take time until the effect of interest can be measured. In the case at hand, this is true for the quality aspect postrelease defect density. From the first application of a technology until the release, it might take 6 months. The release, however, is the earliest time from which on defects might be reported. Furthermore, in this case study, it is only during the processing of a defect that it can be determined whether it belongs to old system parts or to system parts that have been added with the new technology. Thus, the earliest point of study can be 9 months after the introduction of the new technology.

Thus, if for example the effect of the MDD introduction (around the spring of 2009, see time line in Figure 4) on defect density should be measured, defect data from earliest start of 2010 should be considered.

Latest point of study—before-after comparison When comparing 2 technology settings or processes, where one was instantiated initially and is substituted by another one, a before-after comparison is done. In this case, the latest time from which data can be studied is before the next change that concerns the development. This can be any change. For example, in our case study, continuous integration was introduced start of 2013. However, around the same time a lot of process changes happened. Thus, it is not clear whether any differences in productivity or quality found between end of 2012 and end of 2013 would be due to the one or the other changes.

latest point of study—comparison of parallel settings When 2 technology settings are used in parallel, not a before-after comparison is required. This relaxes the latest point of study in theory a bit. This comparison is more robust against changes that happen for development with both technology settings at the same time, eg, the process changes affected both classical C and MDE development.

However, there is nonetheless a latest point of study, which is determined by a change that concerns only one of the development settings. In our study, we can see that a change happens to the classical C setting in 2011 and another one to MDD around the start of 2012.

Summary To sum up, when analyzing the time line, it can be seen that change in a company impacts what data can be used for an analysis tremendously. For example, taking the earliest and latest point of study for a comparison of defect density between the technology settings MDD and classical C, data between 2010 and mid of 2011 might be used. This is in contrast to the actual more than 5 years there both settings are used in parallel.

In general, it is not impossible to study the effect even after the latest point of study. However, the cause-effect relationship will be blurred. Since this might be acceptable (eg, if a change is considered as minor), it is up to the manager or researcher performing the analysis to decide about that.

To conclude, changes impact the usability of data. Given the extend of changes that happen in big companies, we think it is absolutely imperative that any serious research performs similar investigations of change histories.

7 | THREATS TO VALIDITY

In the following, we discuss threats to the validity of our insights. Therefore, we address construct, internal, external, and conclusion validity as discussed in Runeson and Höst.²⁸

Construct validity Interviews always have threats to construct validity, since, for example, gaps in industry specific term usage need to be bridged. To reduce these threats, we applied grounded theory, where the joined interview at the end of the study served as a verification instance.

Of course, these joined interviews also include threats since hierarchies may have impact on whether people disagree with each other's opinions. This is a threat that we cannot entirely rule out. However, we are in a lucky situation that hierarchy plays in the Swedish society a smaller role than it would in other cultures. Thus, we can at least assume that the extent to which this threat affects our data is slightly smaller than it would have to be expected for a study in Germany or France.

One threat to construct validity concerns the completeness of the time line of changes. On the one hand, the information in the time line is limited to the scope of the study, ie, to changes that concern the context of used modeling environment and thereby applied processes. Thus, the collected data can only show an extract of the changes that were going on. In addition, all collected information stems from the memory of the interviewees, and thus, precise dates of changes cannot be collected. Furthermore, we cannot give a guarantee that the collected information is complete. Especially information about changes that happened further in the past is more likely to get lost during our assessment than more recent changes. We tried to reduce that threat by asking targeted questions, eg, "has it always been like that?". In the end, we can be sure that we did not collect information about changes that did not happen. Thus, we are convinced that the assessed time line provides valuable and valid information.

Internal validity One threat to internal validity that plays a role in our study is the possible selection bias in the set of our interviewees. Again, this cannot be ruled out completely, eg, all of our interviewees are senior staff. Thus, our results probably would not express effects

that a development technology might have on the teams that include a lot of junior staff. However, we selected the interviewees such that they all have experience with both development settings. Thus, we consider that our findings are not biased by an underrepresentation of experiences with 1 of the 2 settings.

External validity As for all case studies, it is hard to predict to what extent our results can be generalized to companies of other sizes or domains. However, with Ericsson, we selected a case study in a large company in the highly relevant domain of telecommunication. Therefore, we believe that our observations nonetheless provide valuable qualitative insights.

However, we expect the assessed time line to be very company specific. Probably the extent to which changes happen is different in smaller companies. Nonetheless, we think that this data is interesting since it can create an awareness of change complexity and hopefully inspires future research.

Conclusion validity Since we focused on in-depth interviews of experts, the collected amount of data is for sure not big enough to make any statements on frequencies of the observed effects. However, we focused here on making qualitative insights instead of quantitative ones. Therefore, we consider this limitation of our data acceptable.

Furthermore, the joined meeting of all interviewees helped to increase the quality of the observations, since all participants had the chance to express whether they disagree with each other on made observations.

8 | CONCLUSION

In this paper, we study the ecosystem of process, technologies, change, and evaluations that surrounds developers. We present a case study on the balance of development technologies and software processes/teams. Furthermore, we investigated how technology and process change over time, how given technology settings are perceived by developers and how they cope with change. Finally, we discussed how the regular change of technology and process impacts the ability to judge even those aspects of the development environment.

Our insight partially confirmed observations that were made in related studies on the impact of technologies on team roles and communication. However, we could also collect some new insights on how the tooling impacts developers' decisions on the timing of small iterations.

The uncovered time line shows that changes in large companies can happen regularly, limiting the amount of quantitative data that can be used to evaluate productivity or quality gains of new tools or methods. Finally, the collected data shows that some known problems of model usage did not occur in the studied company.

In future work, it would be interesting to check for some of the observations whether they can be further generalized, eg, abstracted from model-based code generators to code generators that consume configuration files or even to compilers. Furthermore, it would be interesting to investigate the frequencies with which the observed impacts can be found in industry.

We identified a strategy to gain and spread knowledge about how to use certain artifacts. It would be interesting to study in future whether similar strategies could also be applied to the adoption of process changes within a company.

ACKNOWLEDGMENTS

We want to thank Ericsson for their cooperation in this study. Especially, we are thankful to our interviewees for their participation and the time they invested, as well as for the many interesting insights they provided into their working life. Furthermore, we appreciate the constructive feedback that the Ericsson internal reviewers provided on an early version of this paper.

REFERENCES

- Kalus G, Kuhrmann M. Criteria for software process tailoring: a systematic review. *Proceedings of the 2013 International Conference on Software and System Process*, ICSSP 2013. ACM, New York, NY, USA; 2013:171–180.
- Hebig R, Giese H. On the complex nature of MDE evolution and its impact on changeability. *Software & Systems Modeling*. 2015:1–24.
- Hebig R, Schmied A, Weisemöller I. Lessons learned from co-evolution of software process and model-driven engineering. In: Kuhrmann M, Münch J, Richardson I, Rausch A, Zhang H, eds. *Managing Software Process Evolution - How to Handle Process Change?*, chapter 13: Springer International Publishing 2016:255–280.
- Hebig R, Bendraou R. On the need to study the impact of model driven engineering on software processes. *Proceedings of the 2014 International Conference on Software and System Process*, ICSSP. ACM, New York, NY, USA; 2014:164–168.
- Diaz M, Sligo J. How software process improvement helped Motorola. *IEEE Softw*. 1997;14(5):75–81.
- Ocampo A, Münch J. Rationale modeling for software process evolution. *Softw. Process*. 2009;14(2):85–105.
- Kuhrmann M, Diebold P, Münch J. Software process improvement: a systematic mapping study on the state of the art. *PeerJ Computer Science*. 2016:1–38. <https://doi.org/10.7717/peerj-cs.62>
- Unterkalmsteiner M, Gorschek T, Islam AM, Cheng CK, Permadi RB, Feldt R. Evaluation and measurement of software process improvement a systematic literature review. *IEEE Transactions on Software Engineering*. 2012;38(2):398–424.
- Kuhrmann M, Fernández DM, Ternité T. Realizing software process lines: Insights and experiences. *Proceedings of the 2014 International Conference on Software and System Process*, ICSSP 2014. ACM, New York, NY, USA; 2014:99–108.
- Herrmannsdorfer M, Benz S, Juergens E. Automatability of coupled evolution of metamodels and models in practice. In: Czarnecki K, Ober I, Bruel JM, Uhl A, Völter M, eds. *Model Driven Engineering Languages and Systems*, vol. 5301. Springer Berlin Heidelberg: Lecture Notes in Computer Science; 2008:645–659.
- Durisc D, Staron M, Tichy M, Hansson J. Evolution of long-term industrial meta-models—an automotive case study of autosar. *2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications*. Washington, DC, USA:IEEE Computer Society; 2014:141–148.
- Martinez-Ruiz T, Münch J, García F, Piattini M. Requirements and constructors for tailoring software processes: a systematic literature review. *Software Quality Journal*. 2012;20(1):229–260.
- Kulkarni V, Barat S, Ramteerthkar U. Early experience with agile methodology in a model-driven approach. In: Whittle J, Clark T, Kühne T, eds. *Model Driven Engineering Languages and Systems*, vol. 6981. Springer Berlin Heidelberg: Lecture Notes in Computer Science; 2011:578–590.
- Loniewski G, Armesto A, Insfran E. An agile method for model-driven requirements engineering. In: *The Sixth International Conference on Software Engineering Advances (ICSEA 2011)*; IARIA, Barcelona, Spain; 2011:570–575.
- Asadi M, Ramsin R. Mda-based methodologies: an analytical survey. *Proceedings of the 4th European Conference on Model Driven Architecture: Foundations and Applications*, ECMDA-FA '08. Springer-Verlag, Berlin, Heidelberg; 2008:419–431.
- Aranda J, Borici A, Damian D. Transitioning to model-driven development: what is revolutionary, what remains the same. *Model Driven Engineering Languages and Systems*, vol. 7590. LNCS; 2012: 692–708.
- Heijstek W, Chaudron MRV. The impact of model driven development on the software architecture process. *Proceedings of the 2010 36th EUROMICRO Conference on Software Engineering and Advanced Applications*, SEAA '10. IEEE Computer Society, Washington, DC, USA; 2010:333–341.
- Staron M. Adopting model driven software development in industry? A case study at two companies. In: Nierstrasz O, Whittle J, Harel D, Reggio G, eds. *Model Driven Engineering Languages and Systems*, vol. 4199. Springer Berlin Heidelberg: Lecture Notes in Computer Science; 2006:57–72.
- Burden H, Heldal R, Whittle J. Comparing and contrasting model-driven engineering at three large companies. *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, ESEM '14. ACM, New York, NY, USA; 2014:14:1–14:10.
- Whittle J, Hutchinson J, Rouncefield M. The state of practice in model-driven engineering. *Software, IEEE*. 2014;31(3):79–85.
- Nikitina N, Kajko-Mattsson M. Developer-driven big-bang process transition from Scrum to Kanban. *Proceedings of the 2011 International Conference on Software and Systems Process*. ACM New York, NY, USA; 2011:159–168.
- Whittle J, Hutchinson J, Rouncefield M, Burden H, Heldal R. Industrial adoption of model-driven engineering: are the tools really the problem? In: Moreira A, Schätz B, Gray J, Vallecillo A, Clarke P, eds. *Model-Driven Engineering Languages and Systems*, vol. 8107. Springer Berlin Heidelberg: Lecture Notes in Computer Science; 2013: 1–17.
- Hutchinson J, Whittle J, Rouncefield M, Kristoffersen S. Empirical assessment of MDE in industry. *Proceedings of the 33rd International Conference on Software Engineering*. ACM New York, NY, USA; 2011:471–480.
- Mohagheghi P, Dehlen V. Where is the proof?—a review of experiences from applying MDE in industry. *European Conference on Model Driven Architecture-Foundations and Applications*. Springer-Verlag Berlin Heidelberg; 2008:432–443.
- Forward A, Lethbridge TC. Problems and opportunities for model-centric versus code-centric software development: a survey of software professionals. *Proceedings of the 2008 International Workshop on Models in Software Engineering*. ACM New York, NY, USA; 2008:27–32.
- Staron M. Adopting model driven software development in industry—a case study at two companies. *International Conference on Model Driven Engineering Languages and Systems*. Springer-Verlag Berlin Heidelberg; 2006:57–72.
- Kuhn A, Murphy GC, Thompson CA. An exploratory study of forces and frictions affecting large-scale model-driven development. *Proceedings of the 15th international conference on Model Driven Engineering Languages and Systems*. Berlin Heidelberg: Springer-Verlag; 2012: 352–367.
- Runeson P, Höst M. Guidelines for conducting and reporting case study research in software engineering. *Empirical software engineering*. 2009;14(2):131–164.

29. Seaman CB. Qualitative methods in empirical studies of software engineering. *Software Engineering, IEEE Transactions on*. 1999;25(4):557–572.
30. Corbin JM, Strauss A. Grounded theory research: procedures, canons, and evaluative criteria. *Qualitative sociology*. 1990;13(1): 3–21.
31. Unified modeling language version 2.5. Available from: <http://www.omg.org/spec/UML/2.5/Beta2/PDF/>, September 2013. OMG Document Number: ptc/2013-09-05.
32. Karaila M. Evolution of a domain specific language and its engineering environment—lehman's laws revisited. *Proceedings of the 9th OOPSLA Workshop on Domain-Specific Modeling*. Orlando, Florida; 2009:1–7.

How to cite this article: Hebig R, Derehag J. The Changing Balance of Technology and Process: A Case Study on a Combined Setting of Model-Driven Development and Classical C Coding. *J Softw Evol Proc*. 2017;29:e1863. <https://doi.org/10.1002/smr.1863>